

# Capstone: Build an Interactive App

The single end-of-course assessment. Students demonstrate everything from all six modules by building a small, complete app of their own — a different app from the QuickNote project built together in Lesson 21. Score it with the rubric below; completion requires an app that runs with no console errors.

Covers Modules 1–6

Deliverable: a working app (choice of 3 briefs)

Pass: Proficient + runs error-free

---

## HOW IT RUNS

- ◆ **Kickoff (Day 5, 3:30):** students pick one of the three briefs below, sketch their data (what one saved item looks like as an object), and list their functions — the same planning step as QuickNote.
- ◆ **Build time:** about 4–6 hours of independent work. Set a due date (one week after the course works well) and offer one mid-point check-in.
- ◆ **Rules:** vanilla JavaScript only — no frameworks or libraries. Lesson code and QuickNote **patterns** may be reused; the app itself must be the student's own. Another keyless public API that supports browser requests may be swapped in with your approval.

### ◆ REQUIRED DELIVERABLES

- ◆ A project folder with `index.html`, `style.css`, and `app.js` (zipped, or a repository link).
- ◆ A short `README`: which brief, the features built, and any known issues.
- ◆ A **3-minute demo** (live or recorded): add data, reload the page to show it persisted, trigger one validation error and one API call.

---

## CHOOSE ONE BRIEF

A

### Shelf — a reading list

Open Library API

Search for books, add them to a personal shelf, and track what you're reading.

- ◆ A search form (validated: at least 2 characters after trimming) that fetches `https://openlibrary.org/search.json?q=TERM&limit=10` and lists title, author, and first-published year.
- ◆ An **Add to shelf** button on each result; no duplicates on the shelf.
- ◆ Each shelf book has a status (Want to read · Reading · Done) you can change, and a Remove button.
- ◆ The shelf and statuses survive a page reload; a summary line shows counts per status.

Stretch: filter the shelf by status; show covers from `covers.openlibrary.org/b/id/COVER_ID-M.jpg`.

## B Weather Board — saved cities

Open-Meteo API

Keep a board of cities you care about, each showing its current weather.

- ◆ An add-city form (validated: required, at least 2 characters, not already on the board).
- ◆ Look the city up with `https://geocoding-api.open-meteo.com/v1/search?name=CITY&count=1`; show a clear "City not found" message when there are no results.
- ◆ Fetch current conditions with `https://api.open-meteo.com/v1/forecast?latitude=...&longitude=...&current=temperature_2m,wind_speed_10m` and render one card per city with a Remove button.
- ◆ The city list (name, latitude, longitude) survives a reload, and the weather is re-fetched on load.

Stretch: a °F/°C toggle (`&temperature_unit=fahrenheit`) remembered in storage; a Refresh-all button.

## C Recipe Box — find & save recipes

DummyJSON API

Search a recipe collection, save favorites, and add recipes of your own.

- ◆ A search form (validated: at least 2 characters) that fetches `https://dummyjson.com/recipes/search?q=TERM` and lists name, cook time, and difficulty.
- ◆ A **Save** button on each result; saved recipes appear in "My Box" with a Remove button.
- ◆ An **Add your own** form: name required, at least 3 ingredients (one per line), prep time a whole number from 1 to 600 minutes — each error shown next to its field.
- ◆ My Box (saved and your own recipes) survives a reload; a filter box narrows it as you type.

Stretch: a detail view with ingredients and instructions; sort My Box by time.

## REQUIREMENTS CHECKLIST (EVERY BRIEF)

### ◆ DOM & EVENTS

- ◆ The page is rendered from a data array with `createElement` + `textContent` — user or API text is **never** inserted with `innerHTML`
- ◆ At least three kinds of interaction (e.g. submit, click, input)
- ◆ List buttons handled with **event delegation**
- ◆ An empty state ("Nothing saved yet") when the list is empty

### ◆ STORAGE

- ◆ Saved data stored in `localStorage` with `JSON.stringify`
- ◆ Restored on page load with `JSON.parse`
- ◆ Missing or corrupted storage falls back to an empty list (no crash)
- ◆ Storage updated after every add, change, and remove

### ◆ FORMS & VALIDATION

- ◆ Every input has a linked `<label>`
- ◆ `preventDefault()` on submit; input trimmed before checking
- ◆ Clear error messages next to the field, announced to screen readers (e.g. `aria-live` or `aria-describedby` )
- ◆ Invalid data never reaches the list or storage

### ◆ FETCH & ERRORS

- ◆ `async` / `await` with `try` / `catch`
- ◆ Checks `response.ok` before using the data
- ◆ A visible loading state while waiting
- ◆ A friendly message for "no results", a bad response, and no network

## GRADING RUBRIC

Score each criterion 1–3 (max 18). **Pass = no criterion below 2 (therefore  $\geq 12/18$ ) and the app runs with no console errors.**

| CRITERION                               | DEVELOPING (1)                                                                                                               | PROFICIENT (2)                                                                                                                         | EXEMPLARY (3)                                                                                       |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>DOM &amp; rendering</b><br><i>/3</i> | Hard-coded HTML, or output built by pasting user/API text into <code>innerHTML</code> ; list goes out of sync with the data. | One render function draws the list from the data array with <code>createElement</code> / <code>textContent</code> ; empty state shown. | Clean separation of data and rendering; efficient updates; polished states (loading, empty, error). |

| CRITERION                          | DEVELOPING (1)                                                                                  | PROFICIENT (2)                                                                                                                                        | EXEMPLARY (3)                                                                                                 |
|------------------------------------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Events & interaction<br>/3         | Some controls don't respond, or one listener per item added in a loop that breaks on new items. | Submit, click, and input handled; list buttons use event delegation; every feature in the brief works.                                                | Smooth UX: keyboard-friendly, focus managed after actions, at least one stretch feature.                      |
| Forms & validation<br>/3           | Invalid input accepted, the page reloads on submit, or errors only in <code>alert()</code> .    | Labeled inputs; trimmed, checked input; clear message next to each field; invalid data never saved.                                                   | Real-time feedback as the user types; errors announced to screen readers; duplicates prevented.               |
| Data & persistence<br>/3           | Data lost on reload, or stored as <code>"[object Object]"</code> .                              | Saved with <code>JSON.stringify</code> , restored with <code>JSON.parse</code> on load; storage updated after every change.                           | Safe loading with a fallback for missing or corrupted storage; tidy data model (ids, one save function).      |
| Fetching data<br>/3                | Fetch fails silently, ignores <code>response.ok</code> , or freezes the UI with no feedback.    | <code>async</code> / <code>await</code> + <code>try</code> / <code>catch</code> ; checks <code>response.ok</code> ; shows loading and error messages. | Distinct messages for no results, bad responses, and no network; the button is disabled while a request runs. |
| Code quality & accessibility<br>/3 | Console errors; global clutter; unclear names; unlabeled controls.                              | Runs with no console errors; <code>const</code> / <code>let</code> , small named functions, comments where useful; labels and visible focus.          | Well-organized, readable code; semantic HTML; good contrast; a README a stranger could follow.                |

#### COMPLETION POLICY

A student **completes the course** when their project meets the pass rule: **no criterion below 2 (therefore  $\geq 12/18$ ) and the app runs with no console errors**. A 12/18 total with one criterion at 1 does *not* pass. Below the bar, give targeted feedback tied to the criteria above and allow a resubmission.

### GRADING TIP: TEST LIKE A USER

Open the app with the console visible. Submit the empty form, add two items, change one, reload, remove one, reload again. Then go offline (DevTools → Network → **Offline**) and try a search. Five minutes of this covers every criterion. If an API is down or rate-limited on grading day, judge the error handling — that is exactly what it's for.

## SCORING SHEET

| CRITERION                                            | SCORE (1-3) | NOTES |
|------------------------------------------------------|-------------|-------|
| DOM & rendering                                      |             |       |
| Events & interaction                                 |             |       |
| Forms & validation                                   |             |       |
| Data & persistence                                   |             |       |
| Fetching data                                        |             |       |
| Code quality & accessibility                         |             |       |
| Total /18 · Brief (A/B/C) · No console errors (Y/N): |             |       |